Docker Notes

Your Name

Date:

Contents

Chapter 1

Notes info introduction

Purpose of notes These are a set of notes on Docker.

Chapter 2

Geting Started with Docker

2.1 Base CLI Command

2.1.1 Docker Image commands

```
docker image build
docker image history
docker image inspect
docker image pull
```

2.1.2 Docker Container Commands

```
docker container create
docker container exec
docker container logs
docker container run
```

Can use man pages via CLI. See `docker container --help` See `docker container create --help`

Consider old syntax vs new syntax

```
docker run <image>
docker ps
docker logs <ps>
#--------------New syntex
docker container run
docker container ls
docker contiainer logs <id>
```

`docker ps` is the same as `docker container ls`

2.2 Images vs Containers

A Docker image is a standalone, read-only package that includes everything needed to run a software application: the code, runtime, system tools, libraries and settings.

- App Runtime

- Libraries
- Config and Env Vars
- Dependencies

Docker containers are running instances of images. See docker hub for more

Can launch multiple containers from a Single Docker Image Can launch multiple containers from different docker Images

2.2.1 Attached vs Detached modes

In most cases you will be running containers in detached modes, however there some specific use cases where you might wan tot run it in attached mode as well.

When you use the docker container run ¡image¿ without any extra flags, Docker starts the container in Attached Mode. In this mode, your terminal's local standard input, output and error streams are "attached" to the container.

Note: If you press Ctrl + C in ther terminal, you send a SIGINT signal to the container's primary process, which usually stops the container entirely.

To run a container in the background, you use the -d or –detach flag. This detached mode is the standard for any long-running service, for example web server containers or database containers. Ideally, you would like it to be runnig all of the time so you would use it with detached mode.

```
docker container run nginix
docker container run --detach nginix
docker container run -d nginix
```

Generally in most cases, you wil be using the detached mode in the production environments.

2.2.2 Basic Container Management Commands

When you have container running you can perform multiple operations on the container.

Managing the state of a container

```
docker container start <name>
docker container stop <name>
docker container restart <name>
```

Monitoring Performance

Example: A container can be consuming a certain level of memory and CPU

```
docker container stats <name>
```

This command can be useful in identifying which container is using the most CPU.

```
docker container inspect <name>
```

This command will get detailed information related to the container.

```
docker container rm <name>
docker container rm <name> --force
#--- use for exited containers
docker container ls -a
docker container rm <name1> <name2>
```

This will remove a container. If the container is running you can use force remove.

2.2.3 Basic Container Lifecycle

A docker container lives as long as its main process runs. If the process stops, the container stops. If the process is running the container will continue to run.

Short-lived containers - Use for scripts, batch jobs and one time commands.

Long Running Containers - Run a continuous process User for Web Servers, APIs, Database etc.

In production environments you will most likely use long-running containers.

Default Container Command

In Docker, the default container command is the instruction that tells the container process which process to run immediately upon starting. You can override the default container command if required.

2.2.4 Docker Container exec

`docker exec` lets you run a command inside an already running container.

```
docker container exec nginx ls -l
```

2.2.5 docker container exec - important of -i and -t flags

For simple commands that just print the output, you don't need any flags. The command runs, prints its result to your terminal and exits. No interaction is needed.

In most cases however you would want ot go inside the container, so you will require a shell. In such cases, you will require an additional set of flags that need to be added here.

If you want to connect inside a container using a shell like bash, sh (similar to SSH), the default approach will not work. Bash will close immediately. This is because it is an interactive shell that detects there is no input stream (STDIN) to read from. Without an active input stream or a terminal, it assumes its job is done and exists immediately.

2.2.6 The -i flag (interactive)

By default, Docker runs commands in non-interactive mode and does not keep the input stream (STDIN) open. using -i keeps STDIN attached to the container process, which is necessary for any interactive process.

```
docker container exec -i nginx bash
```

2.2.7 The -t Flag (TTY)

This will give you a full-fledged experience. Overview: the hyphen t flag allocates a pseudo terminal. If you just use the -t flag, the standard input get closed so that the keystrokes have no path to reach the process inside the container.

```
docker container exec -t nginx bash # This is very similar to SSH into a Linux
    server
```

Using the -it option, users can get a full interactive shell. You can type, navigate and explore.

```
docker container exec -it nginx bash
docker container exec -i -t nginx bash #Equivalent
exit # Return to the host system.
```

Using this command, you can run everything inside the container environment.

Note: Not every container comes with a shell, however the majority of them do. Note: Navigate Linux/OS file directory to see which binaries are available.

2.3 Basics of Ports

When data arrives at your computer or a server, the operating system needs to know which program should receive it. Ports help figure out which application should receive incoming data. An application can listen on a specific port number (port binding)

When incoming data comes, this data will also have the port number. The server will forward the data to the application with the matching port number. Ports are numbers from 0 to 65,535 and only one process can bind to a specific port on the same IP address at a time.

It is the responsibility of the incoming data to provide the port number so that the operating system can correctly map and send the data to the appropriate program over here.

```
netstat -ntlp # for linux. See what applications are bound to which ports.
```

2.4 Publishing Ports in Docker

When a container has been started, it is assigned a unique IP address within its configured Docker network. The application running inside the container may listen on a specific port.

Containers connected to the same network can communicate with each other using the contain's IP address and the application's listening port.

Note: The internal Docker network IP addresses are not directly accessible from the host system (In Windows and Mac OS). On linux hosts, the Docker bridge network is directly reachable from the host via container IP.

```
curl 172.17.0.2:80 # This will work on Linux not on windows or mac OS
```

2.4.1 Ports Concept

When we talk about ports in Docker, we are always talking about two different locations. - Host Port (outside) - The port you type into your browser (e.g. localhost:8080) - Container Port (inside) - The port the actual application is configured to use (e.g. 80)

2.4.2 Connecting to Container Application

To allow a container to accept incoming connections from the host or the outside world, you must bind (map) it to a host port. Once this is done the users can send a request to the IP address of the server followed by the port number.

```
docker container run -d --name nginx -p 8080:80 nginx
curl 172.17.0.2:8080 # this should work after the above command
```

2.5 Docker Restart Policy

by default, Docker containers will no start when they exit or when the docker daemon is restarted. Sometimes the server restarts and we want all the the containers to automatically be started again.

Docker provides restart policies to control whether your containers start automatically when they exit, or when Docker restarts.

2.5.1 Restart policies

You can specify the restart policy by using the –restart flag with the docker run command.

- no - The defaul. Docker will not restart the container under any circumstances
- on-failure - Restarts only if the container exits with a non-zero exit code
- unless-stopped - Always restarts except when you manually stop the container. If manually stopped, it won't restart even after a daemon restart.
- always - Always restarts regardless of exit code, including after a daemon restart — even if the container way manually stopped before the daemon went down.

In production environments we want the containers to start back up again should the server restart.

2.5.2 Choosing the right policy

Policy	Restarts on crash?	Restarts on success exit?	Restarts after daemon restart	
no	No	No	No	Deve
always	Yes	Yes	Yes	Critical se
on-failure	yes	no	Only if it was running	Bat
unless-stopped	Yes	Yes	Only if no manually stopped	Databases

Chapter 3

Creation, Management and Registry

3.1 Basics of a Dockerfile

A docker container is a running instance of image, whatever container you make is based of one of these docker images. There are numerous images available. From these images, multiple containers can be created.

For an organisation, you may want your own customized images.

A dockerfile is a simple text file containing ordered step-by-step instructions (commands) used to build a Docker image.

The workflow is, docker file into docker image into a running docker container. To create a docker file create a a file called ”Dockerfile”.

```
docker build -t image_name . # will reference the docker file in the same folder
docker container run -d -p 8080:80 --name container_name container_image:latest #
    Create the container
```

There is a wide range of instructions available that can be used in a Dockerfile to build and image.

Some of these include:

Instructions	Description
FROM	The starting point. Every Dockerfile must start with this
CMD	The default command that executes
RUN	Executes a command during the build process (e.g installing a package)
COPY	Copies files from your local machine into the container
SHELL	Set the default shell of an image

3.1.1 Format of a Dockerfile

The format of a Dockerfile follows the syntax INSTRUCTION ¡arguments¿

3.2 Docker file Instructions

Before you can createa a docker file the necessary set of instructions. See Docs for the set of instructions. Using this you can build a custom set of images.

3.2.1 FROM

The FROM instruction sets the base image that all subsequent instructions build upon.

3.2.2 RUN

The RUN instruction executes a command during the build process `RUN apt-get install nginx`

3.2.3 CMD

CMD instruction allows users to provide the default command that executes when the container starts.

```
1   FROM python:3.12-slim
2
3   WORKDIR /app
4
5   COPY requirements.txt .
6   RUN pip install --no-cache-dir -r requirements.txt
7
8   COPY . .
9
10  EXPOSE 8000
11
12  CMD ["python", "app.py"]
```

Listing 3.1: Example Dockerfile

3.3 Docker Image Prune

3.4 Layers of a Docker Image

3.5 Docker Build Cache

3.6 Dockerfile - HEALTHCHECK instruction

3.7 Docker commit

3.8 Flattening Docker Images

3.9 Overview of Docker Registries

3.10 Pushing Images to a Central Repository

3.11 Applying Filter for Docker Images

3.12 Moving Images Across Hosts

Chapter 4

Networking

Chapter 5

Orchestration

Chapter 6

Orchestration using Kubernetes

Chapter 7

Installation and Configuration of Docker EE

Chapter 8

Security

Chapter 9

Storage and Volumes